

TÉCNICAS DE COMPILACIÓN

Compilador para un subconjunto de C++

Implementación en Java con ANTLR4

Dato	Información
Asignatura	Técnicas de Compilación
Trabajo	Trabajo Final
Integrantes	Octavio Douglas Tomás Calzada Ignacio Galvan
Profesor	Francisco Ameri Lopez Lozano
Fecha	12 de junio de 2026
Repositorio	github.com/ignagalvan/PROYECTO_FINAL_TC_CALZADA_DOUGLAS_GALVAN

Informe técnico

RESUMEN

El presente informe describe el diseño e implementación de un compilador educativo para un subconjunto del lenguaje C++. El sistema fue desarrollado en Java y utiliza ANTLR4 para generar el analizador léxico y el analizador sintáctico a partir de una gramática combinada. El flujo incluye lectura del programa fuente, tabla de tokens, construcción y visualización del árbol de parseo, análisis semántico basado en ámbitos, generación de código intermedio de tres direcciones, optimización y exportación de resultados.

La implementación reconoce declaraciones, asignaciones, arreglos básicos, expresiones aritméticas, relacionales y lógicas, estructuras if-else, ciclos while y for, sentencias break y continue, funciones, parámetros, llamadas y retornos. Los diagnósticos se centralizan en un componente de salida que utiliza verde para éxito, amarillo para warnings y rojo para errores.

El proyecto incluye pruebas unitarias para la tabla de símbolos, validaciones semánticas y salida coloreada. Como resultado se obtuvo una arquitectura modular, extensible y adecuada para demostrar las principales fases de un compilador. También se identifican limitaciones actuales, especialmente la ausencia de un AST independiente del árbol de parseo, la validación incompleta de compatibilidad de tipos y la selección manual de una optimización por ejecución.

Palabras clave: ANTLR4, Java, compilador, análisis léxico, análisis sintáctico, análisis semántico, código de tres direcciones, optimización.

ÍNDICE

RESUMEN	2
ÍNDICE	3
1. INTRODUCCIÓN	5
1.1 Objetivo general	5
1.2 Objetivos específicos	5
2. ANÁLISIS DEL PROBLEMA	5
2.1 Subconjunto implementado	5
2.2 Requisitos funcionales	5
2.3 Restricciones	6
3. DISEÑO DE LA SOLUCIÓN	6
3.1 Arquitectura general	6
3.2 Flujo de compilación	6
3.3 Decisiones de diseño	6
4. IMPLEMENTACIÓN	7
4.1 Análisis léxico	7
4.2 Análisis sintáctico	7
4.3 Análisis semántico	7
4.4 Tabla de símbolos	7
4.5 Generación de código intermedio	8
4.6 Optimización	8
4.7 Salidas y diagnósticos	9
5. EJEMPLOS Y PRUEBAS	9
5.1 Programa representativo	9
5.2 Pruebas automatizadas	9
5.3 Casos negativos	9
6. DIFICULTADES Y SOLUCIONES	10
7. RESULTADOS Y LIMITACIONES	10
7.1 Resultados alcanzados	10
7.2 Limitaciones conocidas	11
8. CONCLUSIONES	11
9. REFERENCIAS BIBLIOGRÁFICAS	11

ANEXO A. MANUAL DE USUARIO	12
A.1 Requisitos	12
A.2 Compilación	12
A.3 Ejecución	12
A.4 Archivos generados	12
A.5 Interpretación de mensajes	12
A.6 Selección de optimización	12
A.7 Problemas frecuentes	12

1. INTRODUCCIÓN

Un compilador transforma un programa escrito en un lenguaje fuente en una representación que puede ser validada, optimizada y posteriormente ejecutada o traducida. Este proceso se divide en fases especializadas. La separación facilita la detección precisa de errores y permite que cada componente trabaje con una representación apropiada.

El objetivo del proyecto es aplicar los conceptos de Técnicas de Compilación mediante un sistema funcional para un subconjunto de C++. La herramienta recibe un archivo de texto, reconoce sus tokens, valida su estructura, controla declaraciones y ámbitos, produce código intermedio y aplica una técnica de optimización seleccionada en la aplicación.

1.1 Objetivo general

Diseñar e implementar un compilador modular en Java utilizando ANTLR4, capaz de procesar programas representativos de un subconjunto de C++ y generar diagnósticos y código intermedio.

1.2 Objetivos específicos

- Definir una gramática léxica y sintáctica para el subconjunto seleccionado.
- Reportar errores con línea y columna y mostrar una tabla de tokens.
- Administrar variables, funciones y ámbitos mediante una tabla de símbolos.
- Generar instrucciones de tres direcciones con temporales y etiquetas.
- Aplicar técnicas de optimización sobre el código intermedio.
- Exportar los resultados y facilitar su interpretación mediante colores.

2. ANÁLISIS DEL PROBLEMA

El lenguaje completo C++ posee una complejidad que excede el alcance de un trabajo académico. Por ese motivo se definió un subconjunto que conserva los elementos necesarios para estudiar expresiones, control de flujo, ámbitos, funciones y traducción intermedia.

2.1 Subconjunto implementado

Categoría	Elementos reconocidos
Tipos	int, float, double, char, string, bool y void
Variables	Declaración, inicialización opcional, asignación y acceso
Arreglos	Declaración con tamaño entero y acceso mediante índice
Expresiones	+, -, *, /, %, ==, !=, >, <, >=, <=, &&, , ! y negación
Control	if-else, while, for, break y continue
Funciones	Declaración, parámetros, argumentos, llamadas y return
Salida	cout con uno o varios operandos
Comentarios	Comentarios de línea y de bloque

2.2 Requisitos funcionales

- Analizar el archivo fuente y reconocer tokens válidos.
- Detectar errores léxicos, sintácticos y semánticos.

- Visualizar la estructura sintáctica construida por ANTLR.
- Mantener información de símbolos y ámbitos.
- Generar y guardar código intermedio.
- Optimizar y guardar una segunda versión del código.
- Diferenciar visualmente éxito, warnings y errores.

2.3 Restricciones

La herramienta es educativa y no pretende aceptar todos los programas válidos de C++. La gramática define una sintaxis cerrada, no incluye clases, punteros, referencias, plantillas ni biblioteca estándar completa. La salida generada es código intermedio textual, no código máquina.

3. DISEÑO DE LA SOLUCIÓN

3.1 Arquitectura general

La aplicación sigue una arquitectura por fases. La clase App coordina el flujo y detiene el proceso cuando una etapa crítica encuentra errores. ANTLR4 produce MiLenguajeLexer y MiLenguajeParser a partir de MiLenguaje.g4. Los visitors realizan el análisis semántico y la generación de código.

Fase	Entrada	Componente	Salida
Léxica	Caracteres	MiLenguajeLexer	Tokens
Sintáctica	Tokens	MiLenguajeParser	Árbol de parseo
Semántica	Árbol de parseo	SemanticAnalyzerVisitor	Errores y tabla de símbolos
Código intermedio	Árbol validado	CodigoVisitor / GeneradorCodigo	Tres direcciones
Optimización	Lista de instrucciones	Optimizadores	Código optimizado
Presentación	Resultados	SalidaCompilador	Mensajes ANSI y archivos

3.2 Flujo de compilación

```

archivo fuente
-> lexer -> tabla de tokens
-> parser -> árbol de parseo
-> análisis semántico -> tabla de símbolos
-> generación de tres direcciones
-> optimización
-> archivos de salida y resumen

```

3.3 Decisiones de diseño

- Visitor pattern para separar operaciones de la gramática generada.
- Ámbitos encadenados para resolver nombres locales y globales.
- Temporales t0, t1, ... para almacenar resultados de expresiones.
- Etiquetas L0, L1, ... para representar saltos y estructuras de control.
- Pilas de etiquetas para implementar break y continue en bucles anidados.
- Interfaz común OptimizadorIntermedio para intercambiar estrategias.
- Clase SalidaCompilador para centralizar colores y prefijos de diagnóstico.

4. IMPLEMENTACIÓN

4.1 Análisis léxico

Las reglas del lexer se encuentran en `MiLenguaje.g4`. Las palabras reservadas se declaran antes de `ID` para evitar que `int`, `while` o `return` se reconozcan como identificadores. Los espacios y comentarios se descartan con la acción `skip`. Cada token conserva línea y columna, datos utilizados en los reportes.

App reemplaza los listeners predeterminados de ANTLR por un `BaseErrorListener`. Después ejecuta `fill()` sobre `CommonTokenStream` y muestra tipo, lexema, línea y columna. La gramática incluye `OTRO` como captura de caracteres no reconocidos; actualmente esos casos pueden alcanzar el parser y presentarse como errores sintácticos, aspecto registrado como limitación.

4.2 Análisis sintáctico

La regla inicial programa acepta una secuencia de funciones o sentencias seguida de EOF. Las expresiones están organizadas por precedencia: operadores unarios, multiplicativos, aditivos, relacionales, igualdad, AND y OR. El parser genera un árbol de parseo que se visualiza mediante `TreeViewer` de ANTLR.

```
programa : elemento* EOF ;
elemento : funcion | sentencia ;
sentenciaIf : IF PA expresion PC bloque (ELSE bloque)? ;
sentenciaWhile : WHILE PA expresion PC bloque ;
```

Diferencia técnica: La implementación visualiza el árbol de parseo concreto producido por ANTLR. No existe todavía una jerarquía propia de nodos que constituya un AST independiente.

4.3 Análisis semántico

`SemanticAnalyzerVisitor` recorre el árbol y valida declaraciones duplicadas, uso de variables no declaradas, funciones inexistentes, duplicación de funciones y parámetros, y uso inválido de `break`, `continue` o `return`. Los errores incluyen posición y descripción específica.

- `nivelBucles` controla si `break` y `continue` aparecen dentro de un ciclo.
- `nivelFunciones` controla si `return` aparece dentro de una función.
- Las funciones se registran antes del recorrido completo para permitir llamadas anticipadas.
- Cada bloque crea un `Scope` hijo y restaura el ámbito padre al finalizar.

La infraestructura distingue `SemanticError` de `SemanticWarning` y App posee un canal amarillo para warnings. Sin embargo, la versión actual no agrega reglas que produzcan warnings y tampoco infiere tipos de expresiones para comprobar compatibilidad en asignaciones, operaciones, argumentos o retornos.

4.4 Tabla de símbolos

`Symbol` representa una variable o función mediante nombre, tipo, categoría y nivel de ámbito. `Scope` almacena símbolos en un `LinkedHashMap` y mantiene una referencia al ámbito padre. La resolución comienza en el ámbito actual y continúa hacia el exterior. `SymbolTable` concentra las operaciones de declaración, búsqueda y entrada o salida de ámbitos.

Clase	Responsabilidad
Symbol	Modelo inmutable de variable o función.
Scope	Colección local de símbolos y enlace al ámbito padre.

Clase	Responsabilidad
SymbolTable	API para declarar, resolver y navegar ámbitos.
SemanticAnalyzerVisitor	Aplica reglas semánticas usando la tabla.

La impresión actual de la tabla enumera los símbolos conservados en el ámbito global. Los símbolos locales son válidos durante el recorrido, pero no se almacenan en un historial para mostrarlos al final.

4.5 Generación de código intermedio

CodigoVisitor traduce cada construcción a instrucciones textuales de tres direcciones. GeneradorCodigo administra la lista, crea temporales y etiquetas, imprime el resultado y lo guarda en un archivo.

```
t0 = a * b
t1 = t0 + c
x = t1
if !t2 goto L0
goto L1
L0:
```

Construcción	Representación intermedia
Declaración	declare tipo nombre
Asignación	variable = valor
Expresión	temporal = operando operador operando
Condicional	if !condicion goto etiqueta
Bucle	etiquetas y saltos goto
Función	func nombre:, param, return y endfunc
Llamada	param valor y temporal = call nombre
Salida	print valor

Una limitación detectada es que visitDeclaracion emite la declaración, pero no emite la asignación correspondiente cuando la variable posee un inicializador, por ejemplo `int x = 5`.

4.6 Optimización

El proyecto contiene cuatro implementaciones de OptimizadorIntermedio. Cada una recibe una copia del código original, produce una nueva lista y presenta un resumen.

Técnica	Clase	Descripción
Código muerto	OptimizadorCodigoMuerto	Elimina instrucciones inalcanzables después de goto o return.
Propagación	OptimizadorPropagacionConstantes	Sustituye variables conocidas por valores constantes.
Simplificación	OptimizadorSimplificacionExpresiones	Pliega constantes y aplica identidades algebraicas.
Redundancias	OptimizadorSentenciasRedundantes	Elimina autoasignaciones como <code>x = x</code> .

Aunque existen más de tres técnicas, App instancia solamente OptimizadorCodigoMuerto. Las otras opciones están comentadas y requieren cambiar el código para seleccionarlás. No se ejecuta una cadena de varias optimizaciones en una misma compilación.

4.7 Salidas y diagnósticos

El código intermedio se guarda con el sufijo `_codigo_intermedio.txt` y el resultado optimizado con `_codigo_optimizado.txt`. SalidaCompilador utiliza secuencias ANSI y prefijos uniformes.

Estado	Color	Prefijo
Éxito	Verde	[EXITO]
Advertencia	Amarillo	[WARNING]
Error	Rojo	[ERROR]

5. EJEMPLOS Y PRUEBAS

5.1 Programa representativo

```
int suma(int a, int b) {
    return a + b;
}

int x;
x = suma(2, 3);
if (x > 4) {
    cout << "Resultado: " << x;
}
```

Este ejemplo ejercita función, parámetros, retorno, llamada con argumentos, asignación, comparación, if y salida. El parser genera el árbol, el analizador registra suma y x, y el generador emite temporales, parámetros, call, salto condicional y print.

5.2 Pruebas automatizadas

Suite	Cantidad	Cobertura principal
AppTest	1	Prueba base del proyecto.
SalidaCompiladorTest	3	Colores verde, amarillo, rojo y reset ANSI.
SemanticAnalyzerVisitorTest	6	Declaraciones, ámbitos, funciones, parámetros y errores.
SymbolTableTest	7	Declaración, resolución, duplicados y ámbitos.
Total	17	Ejecución con Maven y JUnit.

La ejecución de `mvn test` finaliza sin fallos: 17 pruebas ejecutadas, 0 fallos, 0 errores y 0 omitidas. Las pruebas actuales se concentran en semántica, tabla de símbolos y colores. Faltan suites específicas para generación de código y para cada optimizador.

5.3 Casos negativos

- Variable duplicada dentro del mismo ámbito.
- Asignación o lectura de una variable no declarada.
- Función llamada sin declaración.

- Función o parámetro duplicado.
- break y continue fuera de un bucle.
- return fuera de una función.
- Errores de sintaxis como punto y coma ausente o paréntesis desbalanceado.

6. DIFICULTADES Y SOLUCIONES

Dificultad	Solución aplicada
Precedencia de operadores	Reglas etiquetadas y ordenadas en la producción expresión.
Ámbitos anidados	Cadena de objetos Scope con búsqueda desde el ámbito local.
break/continue anidados	Pilas independientes de etiquetas de salida y continuación.
Llamadas antes de declarar	Registro previo de todas las funciones en visitPrograma.
Código de control	Temporales y etiquetas generados incrementalmente.
Mensajes dispersos	Centralización en SalidaCompilador y listeners personalizados.
Estrategias de optimización	Interfaz común y clases separadas por técnica.

El trabajo colaborativo se gestionó mediante Git y GitHub. El historial contiene ramas temáticas, pull requests, merges y commits de los integrantes, lo que permitió integrar por etapas la semántica, el código intermedio, las optimizaciones y las salidas coloreadas.

7. RESULTADOS Y LIMITACIONES

7.1 Resultados alcanzados

- Gramática ANTLR4 funcional para el subconjunto definido.
- Tabla de tokens con ubicación de cada lexema.
- Parser y visualización gráfica del árbol de parseo.
- Tabla de símbolos con ámbitos y validaciones semánticas de nombres.
- Código de tres direcciones para expresiones, control y funciones.
- Cuatro clases de optimización disponibles.
- Archivos de código intermedio y optimizado.
- Diagnósticos coloreados y pruebas unitarias.

7.2 Limitaciones conocidas

Limitación	Impacto	Mejora recomendada
No existe AST propio	Se trabaja directamente con el parse tree.	Crear nodos AST y un visitor constructor.
Tipado semántico incompleto	No se detectan incompatibilidades.	Inferir tipos y validar operadores, asignaciones y retornos.
Warnings sin reglas	El canal amarillo rara vez se utiliza.	Advertir variables no usadas o conversiones implícitas.
Una optimización activa	No hay pipeline combinado.	Configurar una cadena de optimizadores.
Inicialización no emitida	int x = 5 no produce x = 5.	Emitir declaración y asignación inicial.
Tabla final solo global	No muestra símbolos locales históricos.	Mantener un registro de ámbitos cerrados.
Pruebas parciales	Riesgo en generación y optimización.	Agregar pruebas unitarias e integración.

8. CONCLUSIONES

El proyecto permitió integrar de forma práctica las fases centrales de compilación. ANTLR4 redujo la complejidad de construir el lexer y el parser, mientras que el patrón Visitor facilitó separar la semántica y la traducción de la gramática.

La solución obtenida procesa un subconjunto significativo de C++, administra ámbitos y genera una representación intermedia comprensible. La organización por componentes permite extender el sistema sin modificar las clases generadas por ANTLR.

La evaluación del código muestra que los objetivos principales fueron alcanzados, pero también que la completitud formal requiere un AST propio, verificación de tipos y una integración automática de varias optimizaciones. Estas mejoras constituyen la evolución natural del compilador.

9. REFERENCIAS BIBLIOGRÁFICAS

- Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2007). Compilers: Principles, Techniques, and Tools (2.ª ed.). Pearson.
- Parr, T. (2013). The Definitive ANTLR 4 Reference. Pragmatic Bookshelf.
- ANTLR. (s. f.). ANTLR Documentation. <https://www.antlr.org/>
- Oracle. (s. f.). Java Platform Documentation. <https://docs.oracle.com/en/java/>
- Apache Maven Project. (s. f.). Maven Documentation. <https://maven.apache.org/>
- Repositorio del proyecto.
https://github.com/Ignagalvan/PROYECTO_FINAL_TC_CALZADA_DOUGLAS_GALVAN

ANEXO A. MANUAL DE USUARIO

A.1 Requisitos

- Java Development Kit 8 o superior.
- Apache Maven.
- Sistema operativo con consola compatible con colores ANSI.
- Entorno gráfico disponible si se desea abrir TreeViewer.

A.2 Compilación

```
cd demo
mvn clean package
```

A.3 Ejecución

```
java -jar target/demo-1.0-jar-with-dependencies.jar programa.txt
```

A.4 Archivos generados

- programa_codigo_intermedio.txt: instrucciones antes de optimizar.
- programa_codigo_optimizado.txt: resultado de la técnica seleccionada.

A.5 Interpretación de mensajes

Mensaje	Significado	Acción
[EXITO] en verde	La fase terminó correctamente.	Continuar con la siguiente fase.
[WARNING] en amarillo	Situación no crítica.	Revisar el código fuente.
[ERROR] en rojo	La compilación no puede continuar.	Corregir línea, columna y causa indicada.

A.6 Selección de optimización

En la versión actual la técnica se selecciona en App.java instanciando una implementación de OptimizadorIntermedio. Solo una línea debe quedar activa. La configuración predeterminada utiliza OptimizadorCodigoMuerto.

A.7 Problemas frecuentes

Problema	Causa probable	Solución
No se encuentra el archivo	Ruta incorrecta.	Usar una ruta válida o ejecutar desde la carpeta del archivo.
Error sintáctico	Falta ;, paréntesis o llave.	Revisar la posición indicada.
Variable no declarada	Uso antes de declarar o fuera de ámbito.	Declarar la variable en un ámbito visible.
No aparece TreeViewer	Entorno sin interfaz gráfica.	Usar una sesión de escritorio o desactivar la visualización.
Colores no visibles	Terminal sin ANSI.	Ejecutar en Windows Terminal, PowerShell moderno o terminal compatible.